PROGRAMMATION ORIENTEE OBJECT POO

1. classe et objet:

- ### Définition :

Une classe est un modèle (plan) pour créer des objets. Un objet est une instance (exemple concret) de cette classe.

- ### Explication :

La classe définit :

- ▢ des attributs (variables)
- ▢ des méthodes (fonctions)

- ### Exemple :

```
class Personne:
    nom = "Ali"
    age = 20
p1 = Personne()
print(p1.nom)
print(p1.age)
```

2. Méthode __init__ (constructeur):

- **Définition :**

C’est une fonction spéciale appelée automatiquement lors de la création d’un objet.

- **Explication :**

Permet d’initialiser les valeurs.

- **Exemple :**

```
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
p1 = Personne("Ali", 20)
print(p1.nom)
print(p1.age)
```

3. Attributs et Méthodes:

- **Définition :**

Attribut → donnée de l’objet

Méthode → action de l’objet

- Exemple :

```
class Voiture:
    def __init__(self, marque):
        self.marque = marque
    def afficher(self):
        print("Marque :", self.marque)
v1 = Voiture("Toyota")
v1.afficher()
```

4. Le mot-clé self:

- Définition :

self représente l'objet lui-même.

- Explication :

Permet d'accéder aux attributs et méthodes de l'objet.

- Exemple :

```
class Etudiant:
    def __init__(self, nom):
        self.nom = nom
    def saluer(self):
        print("Bonjour", self.nom)
```

e1 = Etudiant("Ali")

e1.saluer()

5. Encapsulation (protection des données) :

- ### Définition :

C’est le fait de cacher les données pour les protéger.

- ### Explication :

On utilise _ ou __ devant le nom.

- ### Exemple :

```
class Compte:
    def __init__(self, solde):
        self.__solde = solde
    def afficher(self):
        print(self.__solde)
c = Compte(1000)
c.afficher()
```

6. Héritage:

- ### Définition :

Une classe peut hériter d’une autre.

- Explication :

Permet de réutiliser le code.

- Exemple :

```
class Animal:
    def parler(self):
        print("L'animal parle")
class Chien(Animal):
    def aboyer(self):
        print("Le chien aboie")
c = Chien()
c.parler()
c.aboyer()
```

7. Polymorphisme:

- Définition :

Permet d’utiliser une même méthode de différentes façons.

- Exemple :

```
class Chat:
    def parler(self):
```

```
        print("Miaou")
class Chien:
    def parler(self):
        print("Wouf")
for animal in [Chat(), Chien()]:
    animal.parler()
```